Don't Be An Iter-Hater!

Iterators in Go

Scott Nicholas Allan Smith

London Gophers

2025-04-16

About Me

Soon: Elixir & React at Doccla (fingers crossed)

Currently: Clojure at Riverford Organic Farmers

Previously: Go, Python, Java, C, etc...

Big fan of functional programming (patterns/approaches)

Growing towards lead/staff principal roles

Aspiring conference speaker

Where Are We Going?

- Motivation
  - What Are Iterators?
  - In Other Languages
- Go Iterator Basics
  - Range Over Functions, `iter.Seq` & `iter.Seq2`
  - Reimplementing Range Over Slices, Maps
  - Anatomy Of/Patterns In A Go Iterator
- Examples
  - Iterating Over a Custom Type (`Set`)
  - Infinite Sequences

Iterators

A standard way of "going through" a sequence of values

Usually all they provide is

- some way of getting the next element (if any)
- some way of knowing if there is another element to get

Other Languages

At least these languages have an iterator concept

- Java (`java.util.Iterator`)
- Javascript ("iterator protocol")
- Rust (`std::iter::Iterator`)
- Python (`__iter__` / `__next__`)
- Elixir (`Enumerable`)
- Clojure (`ISeq`)

Other Languages

Language	Iterator	Next	Finished?
Java	`java.util.Iterator`	`next()`	`hasNext() == false`
Javascript	iterator protocol	`next().value`	`next().done`
Rust	`std::iter::Iterator`	`next()`	`next() == None`
Python	`__iter__` / `__next__`	`__next__`	Raises `StopIteration`

Elixir & Clojure - it's surprisingly complicated...

Iterators In Go

"Iterators" in Go are really just special functions

One of three forms

```
func(yield func() bool)

func(yield func(k K) bool) -> iter.Seq[K]

func(yield func(k K, v V) bool) -> iter.Seq2[K, V]
```

Range Over Functions

The "official" name of the iterator feature

```
iter0 := NewZeroArgumentIterator()
for range iter0 {}

iter1 := NewOneArgumentIterator()
for i := range iter1 {}

iter2 := NewTwoArgumentIterator()
for k, v := range iter1 {}
```

Yield Function

That's a function which accepts a function, `yield`, which

- accepts 0, 1 or 2 values which are assigned to loop variables
- returns a boolean which communicates whether another value is required
  - `true` -> another value please
  - `false` -> no more values please

Reimplementing Range Over Slice

Demo

Reimplementing Range Over Slice

```
func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
  i := 0
  return func(yield func(int) bool) {
    for {
      if i >= len(c) {
        return
      }
      anotherValue := yield(i)
      if !anotherValue {
        return
      }
      i++
    }
  }
}
```

Remember, one value `for` over a slice iterates over indices.

Anatomy Of An Iterator

```go
func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
  i := 0 // Vars keeping track of "already iterated over"
         // These can be inside or outside of the anonymous function
  return func(yield func(int) bool) {
    for {
      if i >= len(c) { // - Natural exit condition for finite sequences
        return         // | May be omitted for infinite sequences
      }                // -
      anotherValue := yield(i) // Yield, passing value(s) to the for-range
      if !anotherValue { // -
        return           // | Exit when the loop requires no more values
      }                  // -
      i++
    }
  }
}
```

Anatomy Of An Iterator

Your iterator functions will generally follow this pattern

Always have

- A call to `yield` with the next value to iterate over
- An `if` -> `return` / `break` on `yield`'s return value

Almost certainly have

- Some way to keep track of what's been yielded
- Its own natural exit condition

Where Are We In The Code?

Can be tricky to know who calls who, who's "controlling" execution

Illustrative sequence of events & call stack

This is to help understand, this is not the *real* call stack!

I'm sure the runtime/compiler is doing something much cleverer

Illustrative Sequence / Call Stack

`RangeOverSliceOneArityMain`

Enter for range

`RangeOverSliceOneArityMain > OneArgumentSliceIterator`

Illustrative Sequence / Call Stack

```
func rangeOverSliceOneArityMain() {
    for i := range OneArgumentSliceIterator([]string{"Amos", "Naomi", "Alex", "Holden"}) {
        println(i)
    }
}

func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
    return func(yield func(int) bool) {
        i := 0
        for {
            if i >= len(c) {
                return
            }
            anotherValue := yield(i)
            if !anotherValue {
                return
            }
            i++
        }
    }
}
```

Illustrative Sequence / Call Stack

`RangeOverSliceOneArityMain`

Enter for range

`RangeOverSliceOneArityMain > OneArgumentSliceIterator`

(Anonymous) iterator function returned

Iterator function called

`RangeOverSliceOneArityMain > iterator`

Illustrative Sequence / Call Stack

```
func rangeOverSliceOneArityMain() {
    for i := range OneArgumentSliceIterator([]string{"Amos", "Naomi", "Alex", "Holden"}) {
        println(i)
    }
}

func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
    return func(yield func(int) bool) {
        i := 0
        for {
            if i >= len(c) {
                return
            }
            anotherValue := yield(i)
            if !anotherValue {
                return
            }
            i++
        }
    }
}
```

Illustrative Sequence / Call Stack

...

(Anonymous) iterator function returned

Iterator function called

`RangeOverSliceOneArityMain > iterator`

Iterator calls `yield` (with value `0`)

`RangeOverSliceOneArityMain > iterator > yield`

`i` assigned (value `0`), `for` loop body executes

Illustrative Sequence / Call Stack

```
func rangeOverSliceOneArityMain() {
    for i := range OneArgumentSliceIterator([]string{"Amos", "Naomi", "Alex", "Holden"}) {
        println(i)
    }
}

func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
    return func(yield func(int) bool) {
        i := 0
        for {
            if i >= len(c) {
                return
            }
            anotherValue := yield(i)
            if !anotherValue {
                return
            }
            i++
        }
    }
}
```

Illustrative Sequence / Call Stack

...

`RangeOverSliceOneArityMain > iterator`

Iterator calls `yield` (with value `0`)

`RangeOverSliceOneArityMain > iterator > yield`

`i` assigned (value `0`), `for` loop body executes

`for` loop body completed

`yield` returns

`RangeOverSliceOneArityMain > iterator`

Illustrative Sequence / Call Stack

```
func rangeOverSliceOneArityMain() {
    for i := range OneArgumentSliceIterator([]string{"Amos", "Naomi", "Alex", "Holden"}) {
        println(i)
    }
}

func OneArgumentSliceIterator[T any](c []T) iter.Seq[int] {
    return func(yield func(int) bool) {
        i := 0
        for {
            if i >= len(c) {
                return
            }
            anotherValue := yield(i)
            if !anotherValue {
                return
            }
            i++
        }
    }
}
```

Illustrative Sequence / Call Stack

...

`i` assigned (value `0`), `for` loop body executes

`for` loop body completed

`yield` returns

`RangeOverSliceOneArityMain > iterator`

Iterator calls `yield` (with value `1`)

`RangeOverSliceOneArityMain > iterator > yield`

`i` assigned (value `1`), `for` loop body executes

... ad nauseum

Reimplementing Range Over Slice (Two Valued)

Reimplementing Range Over Slice (Two Valued)

```go
func TwoArgumentIterator[T any](c []T) iter.Seq2[int, T] {
  i := 0
  return func(yield func(int, T) bool) {
    for {
      if i >= len(c) {
        return
      }
      // Only difference is here, we call yield with two values
      anotherValue := yield(i, c[i])
      if !anotherValue {
        return
      }
      i++
    }
  }
}
```

Reimplementing Range Over Map

Demo

Reimplementing Range Over Map

```
// Special case for string keys
func MapIterator[T any](m map[string]T) iter.Seq2[string, T] {
  ks := keys(m)
  i := 0
  return func(yield func(string, T) bool) {
    for {
      if i >= len(ks) {
        return
      }
      anotherValue := yield(ks[i], m[ks[i]])
      if !anotherValue {
        return
      }
      i++
    }
  }
}
```

Aside: Keys From Map Without Range

```
func keys[T any](m map[string]T) []string {
  vs := reflect.ValueOf(m).MapKeys()
  i := 0
  ks := make([]string, 0)
  for {
    if i >= len(vs) {
      return ks
    }
    ks = append(ks, vs[i].String())
    i++
  }
}
```

Because we need to call a method on the `reflect.Value` depending upon the type of the key, I haven't figured out how to generalise this

Eager vs Lazy

In what follows I'm going to talk about *eager* evaluation vs *lazy* evaluation

Roughly:

- *eager* evaluation means stuff is calculated now, even if I don't need it
- *lazy* evaluation means calculation is defered until I need it to happen

Iterating Over a Custom Type (`Set`)

A set is just a collection of elements that contains no duplicates

Lets consider how we can convert a slice to a set

- eagerly
- lazily

"Infinite" Sequences

Not truly infinite - we have bounded memory, power, time, etc...

Seems esoteric

Different natural way of thinking between functional and imperative programming

Functional - sequence (or pipeline) of operations on data

Generate a Random Hex ID

Recent requirement - 16 digit hex id

Look at Go approach

Look at natural approach in functional programming

Generate a Random Hex ID

Demo

Compared To A Functional Language

Elixir

```elixir
fn -> "ABCDEF0123456789"
      |> String.graphemes()
      |> Enum.random() end
|> Stream.repeat()
|> Stream.take(16)
|> Enum.join()
```

Go

```go
chars := []rune("ABCDEFG0123456789")
rc := func() string {
  n := rand.Int31n(16)
  return string(chars[n])
}
r := repeatedly(rc)
t := take(16, r)
id := join(t)
return id
```

Compared To A Functional Language

Clojure

```clojure
(->> #(rand-nth "ABCDEF0123456789")
     (repeatedly)
     (take 16)
     (clojure.string/join))
```

Go

```go
chars := []rune("ABCDEFG0123456789")
rc := func() string {
  n := rand.Int31n(16)
  return string(chars[n])
}
r := repeatedly(rc)
t := take(16, r)
id := join(t)
return id
```

Functional Approach

I would argue that the functional approach

- separates flow control and data transformation more nicely
- reads more like natural language
- therefore can be faster to understand

Your mileage may vary - come discuss after the talk!

Thanks!

For listening and for your questions (shortly!)

Please give brutally honest feedback and criticism

Do you think this with more demos would make a good GopherCon talk?

Come tell me (yes or no!)

---

uk.linkedin.com/in/scottnasmith | github.com/snasphysicist/

Links, projects and thoughts at my website: www.snas.pw

(Run as cheaply as possible with no guarantees on uptime)